Rockchip CAN Developer Guide

ID: RK-KF-YF-C44

Release Version: V1.3.0

Release Date: 2025-10-09

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

Trademark Statement

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

Product Version

Chipset	Kernel Version
RV1126	4.4 & 4.19
RK3568	4.19 & 5.10
RK3588	5.10
RK3562	5.10
RK3506	6.1
RK3576	6.1
RV1126B	6.1

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Date	Author	Version	Change Description
V1.0.0	Elaine	2021-01-26	First version release
V1.1.0	Elaine	2022-11-26	Added RK3568 RK3588
V1.2.0	Elaine	2024-03-26	Added RK3562
V1.2.1	Elaine	2025-04-17	Added Document ID, modified wording
V1.3.0	Elaine	2025-10-09	Rename can driver, Added RK3576/RK3506/RV1126B

Contents

Rockchip CAN Developer Guide

1. CAN Driver
   - 1.1 Driver Files
   - 1.2 DTS Node Configuration
   - 1.3 Kernel Configuration
2. CAN Communication Testing Tools
3. CAN Common Command Interface
4. Troubleshooting Common CAN Issues
   - 4.1 Inability to Send or Receive
   - 4.2 Intermittent Transmission Failure
5. CAN Bit Rate and Sampling Point Calculation

1. CAN Driver

1.1 Driver Files

Location of driver files:

For RV1126/RV1109:

```
drivers/net/can/rockchip/rockchip_can.c
```

For RK3568/RK3588:

```
drivers/net/can/rockchip/rk3568_can.c
```

For RK3562:

```
drivers/net/can/rockchip/rk3562_can.c
```

For RK3576/RK3506/RV1126B:

```
drivers/net/can/rockchip/rk3576_can.c
```

1.2 DTS Node Configuration

Primary Parameters:

- `interrupts = <GIC_SPI 100 IRQ_TYPE_LEVEL_HIGH>;`

  Completion of conversion generates an interrupt signal.

- `clock`

```
    assigned-clocks = <&cru CLK_CAN>;
    assigned-clock-rates = <200000000>;
    clocks = <&cru CLK_CAN>, <&cru PCLK_CAN>;
    clock-names = "baudclk", "apb_pclk";
```

The clock frequency can be modified. If the CAN bit rate is 1M, it is recommended to change the CAN clock to 300M for more stable signals. For bit rates below 1M, a clock setting of 200M is sufficient. The CAN clock should ideally be set to an even multiple of the bit rate for precise frequency division.

- `compatible`

```
    .compatible = "rockchip,can-1.0",
```

For RV1126/RV1109, use "rockchip,can-1.0".

For RK3568, use "rockchip,rk3568-can-2.0".

For RK3588, use "rockchip,can-2.0".

For RK3562, use "rockchip,rk3562-can".

For RK3576/RK3506, use "rockchip,rk3576-can".

For RV1126B, use "rockchip,rv1126b-can".

- `pinctrl`

```
&can {
    pinctrl-names = "default";
    pinctrl-0 = <&canm0_pins>;
    status = "okay";
};
```

Configure `can_h` and `can_l` iomux for CAN functionality use.

- `rx-max-data`

```
&can0 {
    rockchip,rx-max-data = <4>;
};
```

Configures the maximum number of bytes that can be received by rx. If only CAN frames are used, it can be set to 4 (rx byte supports 8), allowing the rx FIFO depth to be 64. If CANFD frames are used, it can be set to 18 (rx byte supports 64), resulting in an rx FIFO depth of 14 frames.

- `auto-retx-cnt`

```
&can0 {
    rockchip,auto-retx-cnt = <100>;
};
```

Configures the number of times tx will automatically retry after a transmission failure. If not configured, it defaults to no limit, continuously retrying. Configure a value between 1 and 0xffff, and the retries will stop after reaching the threshold, configure as needed.

- `dma`

```
&can0 {
    dmas = <&dmac0 20>;
    dma-names = "rx";
};
```

rx has DMA functionality, but if DMA is not desired, the above property can be removed. RV1126B support dma.

1.3 Kernel Configuration

```
Symbol: CAN_ROCKCHIP [=y]
                  |
  | Type  : tristate
                    |
  | Prompt: Rockchip CAN controller
                    |
  |   Location:
                    |
  |     -> Networking support (NET [=y])
                    |
  |       -> CAN bus subsystem support (CAN [=y])
                    |
  |         -> CAN Device Drivers
                    |
  |           -> Platform CAN drivers with Netlink support (CAN_DEV [=y])
                    |
  |   Defined at drivers/net/can/rockchip/Kconfig:1
                    |
  |   Depends on: NET [=y] && CAN [=y] && CAN_DEV [=y] && ARCH_ROCKCHIP [=y]
```

```
Symbol: CAN_RK3568 [=y]
                                  |
  | Type  : tristate
                                        |
  | Prompt: Rk3568 CAN controller
                                     |
  |   Location:
                                        |
  |     -> Networking support (NET [=y])
                                        |
  |       -> CAN bus subsystem support (CAN [=y])
                                        |
  |         -> CAN Device Drivers
                                        |
  |           -> Platform CAN drivers with Netlink support (CAN_DEV [=y])
                                        |
  |   Defined at drivers/net/can/rockchip/Kconfig:10
                                        |
  |   Depends on: NET [=y] && CAN [=y] && CAN_DEV [=y] && ARCH_ROCKCHIP [=y]
```

```
CONFIG_CAN_RK3562=y
```

```
CONFIG_CAN_RK3576=y
```

2. CAN Communication Testing Tools

Canutils is a commonly used package of CAN communication testing tools, which includes 5 independent programs: canconfig, candump, canecho, cansend, and cansequence. A brief description of the functions of these programs is as follows:

`canconfig`

Use to configure the parameters of the CAN bus interface, mainly the baud rate and mode.

`candump`

Receive data from the CAN bus interface and prints it in hexadecimal form to the standard output, or outputs it to a specified file.

`canecho`

Retransmit all data received from the CAN bus interface back to the CAN bus interface.

`cansend`

Send specified data to the designated CAN bus interface.

`cansequence`

Automatically repeats incrementing numbers to the specified CAN bus interface, and can also specify the reception mode and check the received incrementing numbers.

`ip`

Configuration of CAN baud rate, functionality, etc.

Note: The busybox also integrates the ip tool, but the one in busybox is a cut-down version. It does not support CAN operations. Therefore, please confirm the version of the ip command (iproute2) before use.

There are detailed compilation instructions for the above tool package on the network. If you are compiling buildroot yourself, you can directly enable the macro to support the above tool package. You can also contact us to obtain it.

```
BR2_PACKAGE_CAN_UTILS=y
BR2_PACKAGE_IPROUTE2=y
```

3. CAN Common Command Interface

1. Query the current network devices:

`ifconfig -a`

2. CAN Start:

Disable CAN:

`ip link set can0 down`

Set the bit rate to 500KHz:

`ip link set can0 type can bitrate 500000`

Print information for can0:

`ip -details -statistics link show can0`

Enable CAN:

`ip link set can0 up`

3. CAN Send:

Send (Standard Frame, Data Frame, ID:123, Data:DEADBEEF):

`cansend can0 123#DEADBEEF`

Send (Standard Frame, Remote Frame, ID:123):

`cansend can0 123#R`

Send (Extended Frame, Data Frame, ID:00000123, Data:DEADBEEF):

`cansend can0 00000123#12345678`

Send (Extended Frame, Remote Frame, ID:00000123):

`cansend can0 00000123#R`

3. CAN Receive:

Enable printing, wait for reception:

```
candump can0
```

4. Troubleshooting Common CAN Issues

4.1 Inability to Send or Receive

Loopback Mode Testing:

After initiating CAN, use the IO input command to enable loopback self-test (the base address should be based on the actual DTS-configured CAN settings):

```
io -4 0xfe580000 0x8415
```

In loopback mode, if cansend is followed by candump being able to receive, it indicates that the controller is functioning properly. In this state, it is necessary to check the following: whether IOMUX is correct; whether the hardware connections are correct; whether the terminal 120-ohm resistor is connected; and whether the conversion chip is normal.

4.2 Intermittent Transmission Failure

Ensure that the bit rate is accurate. The following command can display the actual bit rate and configuration information of the CAN interface. If there is a deviation in the bit rate, it can cause transmission anomalies. It is necessary to adjust the input clock based on the bit rate to achieve an accurate bit rate.

```
ip -details -statistics link show can0
```

Sample point adjustment, the above CAN command will print the current configured sample point, try to ensure that the sample points are consistent across the network. This can ensure the stability of transmission.

5. CAN Bit Rate and Sampling Point Calculation

The CAN architecture currently calculates automatically based on input frequency and bit rate. The rules for sampling points follow the CiA standard protocol:

```
/* Use CiA recommended sample points */
    if (bt->sample_point) {
        sample_point_nominal = bt->sample_point;
    } else {
        if (bt->bitrate > 800000)
            sample_point_nominal = 750;
        else if (bt->bitrate > 500000)
            sample_point_nominal = 800;
        else
            sample_point_nominal = 875;
    }
```

Bit rate calculation formula (detailed principles can be found on Baidu, this section only introduces chip configuration related):

`BitRate = clk_can / (2 *(brq + 1) / ((tseg2 + 1) + (tseg1 + 1) + 1)`

`Sample = (1 + (tseg1 + 1)) / (1 + (tseg1 + 1) + (tseg2 + 1))`

brq, tseg1, tseg2 refer to the BITTIMING register in the CAN's TRM.